

Programmation Fonctionnelle

TD n° 8 : Exceptions et type Option

1 Échauffement

1.1 Listes d'association

Exercice 1 : Écrire une fonction `find : ('a -> bool) -> 'a list -> 'a`, telle que `find p l` renvoie le premier élément de la liste `l` qui satisfait le prédicat `p`. Si aucun élément de la liste ne satisfait `p`, on lève l'exception `Not_found`.

Exercice 2 : Écrire une fonction `find_opt : ('a -> bool) -> 'a list -> 'a option`, telle que `find_opt p l` renvoie `Some x`, où `x` est le premier élément de la liste `l` qui satisfait le prédicat `p`. Si aucun élément de la liste ne satisfait `p`, on renvoie la valeur `None`.

Exercice 3 : Écrire une fonction `assoc : 'a -> ('a * 'b) list -> 'b`, telle que `assoc a l` renvoie la valeur associée à la première occurrence de la clé `a` dans la liste d'association `l`. Si la clé `a` n'apparaît pas dans `l`, on lève l'exception `Not_found`.

Cette fonction ne doit pas être récursive, mais doit déléguer son calcul à la fonction `find`.

Exercice 4 : Écrire une fonction `assoc_opt : 'a -> ('a * 'b) list -> 'b option`, telle que `assoc_opt a l` renvoie `Some b`, où `b` est la valeur associée à la première occurrence de la clé `a` dans la liste d'association `l`. Si la clé `a` n'apparaît pas dans `l`, on renvoie `None`.

Cette fonction ne doit pas être récursive, mais doit déléguer son calcul à la fonction `find_opt`.

1.2 Recherche dans un arbre

On considère le type OCaml suivant, qui représente des arbres binaires dont les *feuilles* sont étiquetées par des entiers :

```
type tree =
  | Leaf of int
  | Node of tree * tree
```

Exercice 5 : Écrire une fonction `path_to_leaf : int -> tree -> char list`, telle que `path_to_leaf n t` renvoie l'encodage d'un chemin dans `t` vers une feuille étiquetée par `n`. Un chemin dans un arbre sera encodé par une liste de caractères `'g'` ou `'d'` (pour gauche et droite). S'il n'y a pas de feuille étiquetée par `n` dans l'arbre, on lève l'exception `Not_found`.

Par exemple le test suivant doit renvoyer la liste `['g'; 'd'; 'g']` :

```
let t0 = Node(Leaf 0, Leaf 0) in
let t1 = Node(Leaf 1, Leaf 0) in
let t2 = Node(t0, t1) in
let t3 = Node(t2, t0) in
path_to_leaf 1 t3
```

2 Expressions arithmétiques

Nous allons maintenant manipuler des expressions arithmétiques, représentées par le type `expression` ci-dessous :

```
type operator = Add | Sub | Mul | Div
type expression =
  | Const of int
  | Var of string
  | Op of operator * expression * expression
```

Par exemple, l'expression $a/(2+x)$ est représentée en OCaml par la valeur suivante de type `expression` : `Op(Div, Var "a", Op(Add, Const 2, Var "x"))`.

2.1 Évaluer une expression

Pour évaluer la valeur d'une expression, nous avons besoin d'un *environnement*, c'est-à-dire d'une liste d'association de type `(string * int) list`, qui associe à chaque nom de variable une valeur entière. Pour gérer les cas d'erreur, on définit les deux exceptions suivantes :

```
exception Undefined_variable of string
exception Division_by_zero
```

Exercice 6 : Écrire une fonction `eval : expression -> (string * int) list -> int`, telle que `eval expr env` évalue la valeur de l'expression `expr`, où la valeur des variables est donnée par l'environnement `env`.

- Si une variable `s` n'est pas définie dans `env`, on lève l'exception `Undefined_variable s`.
- Si une division par zéro est rencontrée, on lève l'exception `Division_by_zero`.

Pour rechercher une variable dans l'environnement, vous pouvez utiliser votre fonction `assoc` (ou `assoc_opt`, au choix) de l'exercice 3.

Par exemple :

```
# let e = Op(Div, Var "a", Op(Add, Const 2, Var "x"));
# eval e [("a", 16); ("x", 2)];;
- : int = 4
# eval e [("a", 16); ("x", -2)];;
Exception: Division_by_zero.
# eval e [("a", 16)];;
Exception: Undefined_variable "x".
```

2.2 Parser une expression

Notre but est maintenant de *parser* une expression arithmétique contenue dans une chaîne de caractères. C'est-à-dire qu'on veut une fonction `parse : string -> expression` qui analyse une chaîne de caractères (par exemple `"a/(2+x)"`) pour renvoyer l'expression associée. On propose ici une solution *ad hoc*; une méthode plus efficace et plus générique sera étudiée au deuxième semestre dans le cours Grammaires et Analyse Syntaxique.

Pour gérer les cas d'erreur, on définit les exceptions suivantes :

```
exception Unknown_operator of char
exception Unmatched_parenthesis of int
exception Unexpected_character of char * int
```

Commençons par quelques fonctions utilitaires :

Exercice 7 : Écrire une fonction `is_alphanumeric : char -> bool` qui dit si un caractère est alphanumérique (un chiffre ou une lettre).

Vous pouvez utiliser la syntaxe `match c with | '0'..'9' ->`

Exercice 8 : Écrire une fonction `op_of_char : char -> operator` qui transforme un caractère ('+', '-', '*', '/') en son opérateur associé (Add, Sub, Mul, Div). Tout autre caractère lève l'exception `Unknown_operator`.

Exercice 9 : Écrire une fonction `find_next_symbol : string -> int -> int`, telle que `find_next_symbol s start` renvoie le premier indice $i \geq \text{start}$ tel que `s.[i]` n'est pas alphanumérique. Si on atteint la fin de la chaîne, on renvoie `String.length s`.

Exercice 10 : Écrire une fonction `find_matching_parenthesis : string -> int -> int`, telle que `find_matching_parenthesis s start` a le comportement suivant :

- on suppose que `s.[start]` est une parenthèse ouvrante, '(' ;
- le but est de renvoyer l'indice de la parenthèse fermante associée dans la chaîne `s` ;
- si aucune parenthèse n'est trouvée, on lève l'exception `Unmatched_parenthesis start`.

Par exemple :

```
# find_matching_parenthesis "(a+b)*((1+c)-3)" 6;;
- : int = 14
# find_matching_parenthesis "(a+b)*((1+c)-3" 6;;
Exception: Unmatched_parenthesis 6.
```

On peut maintenant écrire notre parser. Pour simplifier, on fait les hypothèses suivantes :

1. La chaîne donnée en entrée ne contient pas d'espaces.
2. La chaîne est *entièrement parenthésée* : il n'y a donc pas à se soucier de la priorité de la multiplication sur l'addition.
3. Les noms de variables contiennent uniquement des caractères alphanumériques, avec au moins une lettre. Par exemple, "toto", "a1", "123x" sont des noms de variables valides, mais "my_var" n'est pas accepté.
Ainsi, si une chaîne alphanumérique peut être convertie en entier à l'aide de la fonction `int_of_string`, c'est une constante ; sinon (exception `Failure _`), c'est une variable.

Exercice 11 : Écrire deux fonctions mutuellement récursives :

- `parse_operand : string -> int -> expression * int`
- `parse_expr : string -> int -> int -> expression`

telles que :

- `parse_operand s i` analyse la chaîne `s` à partir de l'indice `i`, et doit extraire une opérande, c'est-à-dire, soit une constante, soit une variable, soit une expression entre parenthèses. Elle renvoie une paire (`e`, `j`) où `e` est l'opérande qui a été lue, et `j` est l'indice du prochain caractère de la chaîne après avoir lu `e`. Si `s.[i]` n'est ni une parenthèse ouvrante, ni un caractère alphanumérique, on lève l'exception `Unexpected_character (s.[i], i)`.

Par exemple,

```
# parse_operand "(a+1)*b" 0;;
- : expression * int = (Op (Add, Var "a", Const 1), 5)
# parse_operand "(a+1)*b" 6;;
- : expression * int = (Var "b", 7)
```

- `parse_expr s i k` analyse la chaîne `s` entre les indices `i` (inclus) et `k` (exclus), et renvoie l'expression associée.

Rappel : comme on a supposé que l'expression est entièrement parenthésée, une expression peut être soit une opérande, soit deux opérandes séparées par un opérateur. Si l'indice `k` n'est pas atteint après avoir lu deux opérandes, on lève l'exception `Unexpected_character` avec pour argument le caractère suivant dans la chaîne et son indice.

Par exemple,

```
# parse_expr "(1+2)*3" 0 7;;
- : expression = Op (Mul, Op (Add, Const 1, Const 2), Const 3)
# parse_expr "1+(2*3)" 0 7;;
- : expression = Op (Add, Const 1, Op (Mul, Const 2, Const 3))
# parse_expr "1+2*3" 0 5;;
Exception: Unexpected_character ('*', 3).
```

Exercice 12 : Écrire une fonction `evaluate : string -> (string * int) list -> unit`, telle que `evaluate s env` parse la chaîne `s` et l'évalue dans l'environnement `env`. Toutes les exceptions doivent être rattrapées pour afficher des messages d'erreur les plus informatifs possibles.

Par exemple,

```
# evaluate "(a*(b+3))/(x-1)" [("a", 2); ("b", 3); ("x", 4)];;
(a*(b+3))/(x-1) = 4
- : unit = ()
# evaluate "(a*(b+3))/(x-1)" [("a", 2); ("x", 4)];;
Error: undefined variable: b
- : unit = ()
# evaluate "(a*(b+3))/(x-1)" [("a", 2); ("b", 3); ("x", 1)];;
Error: division by zero
- : unit = ()
# evaluate "(a*(b+3))/(x-1" [("a", 2); ("b", 3); ("x", 4)];;
Parsing error: unmatched parenthesis at position 10
(a*(b+3))/(x-1
  ↑
- : unit = ()
# evaluate "(a*(b+3))%(x-1)" [("a", 2); ("b", 3); ("x", 4)];;
Parsing error: unknown operator %
- : unit = ()
# evaluate "(a*(-b+3))/(x-1)" [("a", 2); ("b", 3); ("x", 4)];;
Parsing error: unexpected character - at position 4
(a*(-b+3))/(x-1
  ↑
- : unit = ()
```

Exercice 13 : (Bonus) Améliorer votre parser afin de pouvoir analyser des expressions qui ne sont pas forcément entièrement parenthésées.

Par exemple, `evaluate "1+2*3*4-5+(6+8)/7"` [] doit afficher la valeur 22.

Indication. Utiliser trois fonctions mutuellement récursives : `parse_operand` est la même que précédemment ; `parse_product` lit une liste d'opérandes séparées par des caractères `'*'` ou `'/'` ; et `parse_sum` lit une liste de produits, séparés par des caractères `'+'` ou `'-'`.